

Module 4: Lexical Semantics & Embeddings

WordNet Synsets, TF-IDF, Positive PMI, Word2Vec (Skip-Gram SGNS) & GloVe Log-Bilinear Model

Module IV: Deep Learning & Neural Architectures in NLP — Complete 8-Topic Tracker

Topic 36: Convolutional Neural Networks (CNNs) for Text Classification

Topic 38: Long Short-Term Memory (LSTM) Gate Formulations & Memory Cells

Topic 40: Sequence-to-Sequence Encoder-Decoder with Cross-Attention

Topic 42: BERT (Bidirectional Encoder Representations from Transformers)

Topic 37: Recurrent Neural Networks (RNNs) & Vanishing Gradients

Topic 39: The Transformer Architecture (Scaled Dot-Product Self-Attention)

Topic 41: Transfer Learning in NLP (Pre-Training + Task Fine-Tuning)

Topic 43: GPT (Generative Pre-Trained Causal Autoregressive Decoders)

Topic 36 – 38: Sequential Neural Models (CNN, RNN, LSTM)

Neural Model	Mathematical Formulation & Architecture	Key Advantage & Failure Mode
1. CNN for NLP	1D convolution filters slide across embedding matrices capturing local n -gram phrase features: $c_i = f(\mathbf{w}^T \mathbf{x}_{i:i+h-1} + b) \rightarrow$ Max-Pooling over time.	Highly parallelizable; fails to model long-range sequential syntax dependencies.
2. Vanilla RNN	Recurrent hidden state: $\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$. Unrolls across variable sentence lengths.	Models sequential history; severely suffers from Vanishing and Exploding Gradients over > 10 tokens.
3. LSTM (Hochreiter, 1997)	Regulates information flow using 3 Gates + Constant Error Carousel Cell State C_t : Forget Gate f_t , Input Gate i_t , Output Gate o_t .	Maintains long-range gradient highway over hundreds of tokens; sequential processing inhibits GPU scaling.

Complete LSTM Mathematical Gate Formulations:

$$\text{Forget Gate: } \mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

$$\text{Input Gate: } \mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i), \quad \tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$

$$\text{Cell State Update: } \mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$$

$$\text{Output Gate: } \mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o), \quad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t)$$

Topic 39 & 40: The Transformer Architecture (Vaswani et al., 2017)

1. Scaled Dot-Product Attention Equation:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V}$$

Where $\mathbf{Q}$ (Query), $\mathbf{K}$ (Key), and $\mathbf{V}$ (Value) are linear projections of input embeddings. Division by $\sqrt{d_k}$ prevents gradient saturation in large dimensions!

2. Multi-Head Attention (MHA):

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O, \quad \text{head}_i = \text{Attention}(\mathbf{Q} \mathbf{W}_i^Q, \mathbf{K} \mathbf{W}_i^K, \mathbf{V} \mathbf{W}_i^V)$$

Topic 41 – 43: Pre-Trained Transformers: BERT vs. GPT

Feature	BERT (Devlin et al., Google)	GPT Series (Radford et al., OpenAI)
Core Architecture	Transformer Encoder Only (Bidirectional self-attention).	Transformer Decoder Only (Autoregressive causal masked attention).
Pretraining Objective	1. Masked LM (MLM: predicts 15% '[MASK]' tokens). 2. Next Sentence Prediction (NSP).	Standard Left-to-Right Causal Language Modeling: $\max \sum \log P(w_t \mid w_{<t})$
Optimal Tasks	NLU, Text Classification, Named Entity Recognition, Question Answering.	NLG, Creative Writing, Chatbots, Open-Domain Reasoning, Code Generation.

 M4 Active Recall & Exam Questions

Q1. Explain the working of the Scaled Dot-Product Self-Attention mechanism in Transformers with a diagram. (10 Marks)

Self-attention calculates pairwise relational affinities between every pair of tokens in parallel:

- Linear Projections:** Input matrix $\mathbf{X}$ is multiplied by weight matrices $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V$ to generate $\mathbf{Q}, \mathbf{K}, \mathbf{V}$.
- Score Calculation:** Computes dot-product similarity matrix $\mathbf{S} = \mathbf{Q} \mathbf{K}^T$, reflecting how much each token attends to all other tokens.
- Scaling & Normalization:** Divides by $\sqrt{d_k}$ to stabilize gradients and applies row-wise $\text{softmax}(\frac{\mathbf{Q} \mathbf{K}^T}{\sqrt{d_k}})$.
- Weighted Aggregation:** Multiplies attention weights by value matrix $\mathbf{V}$ to output contextualized embeddings!